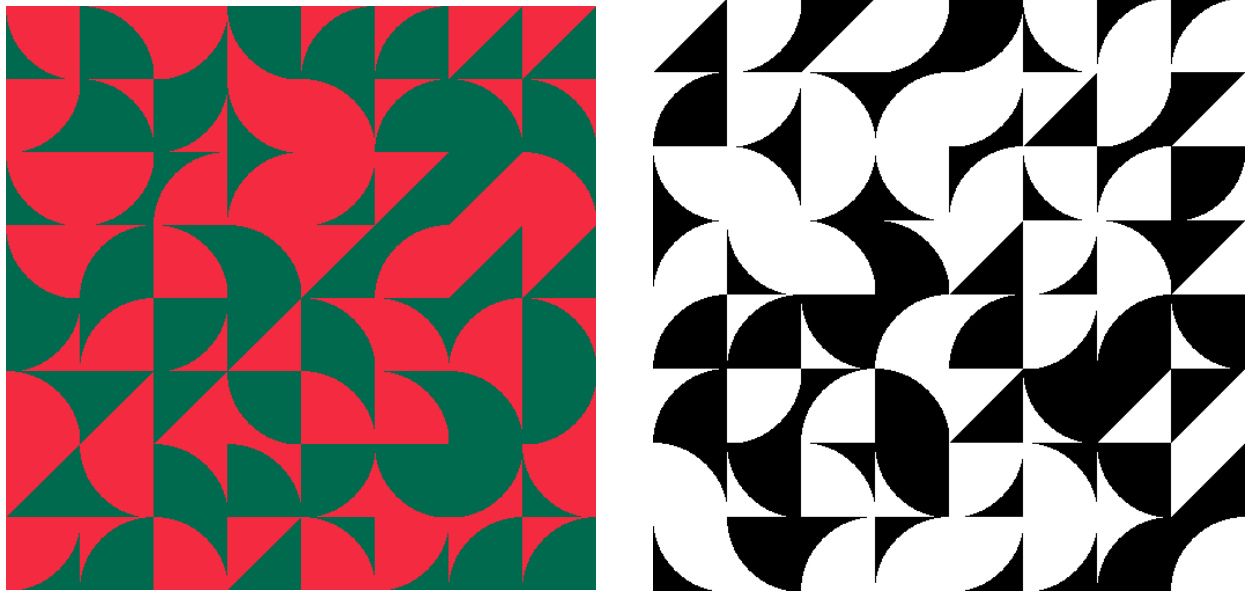


Idea 1)

Leveraging Randomness to Create Art



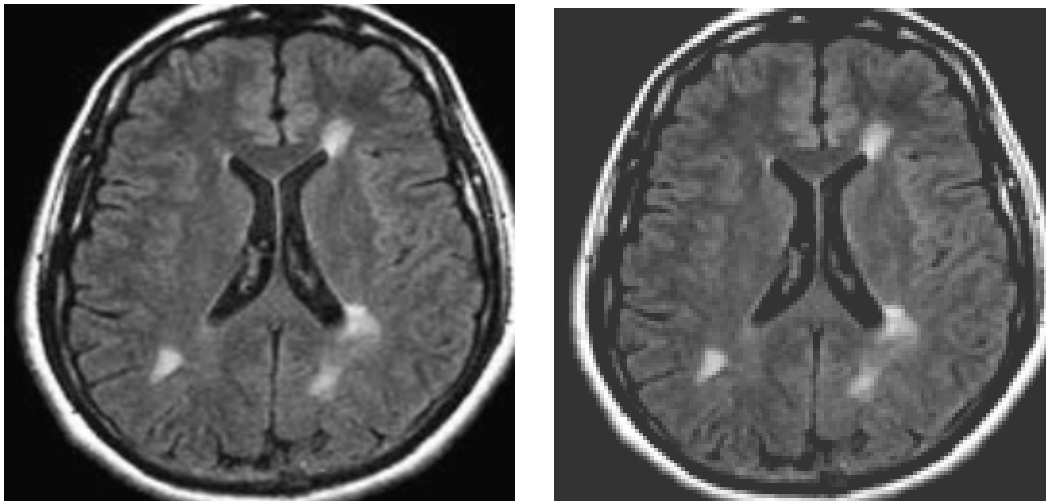
Yes, you can see an animation. Guess what, it is implemented using quantum measurement. We have implemented a quantum circuit of small depth that choose pixel states based on measurement outcome of that circuit. [We are injecting randomness to create artwork.](#)

Implementation Details :inside all_process.ipynb file

Idea : Define state with little images,dynamically choose the states based on circuit measurement result we are selecting states for each cell. The red – green color scheme is taken from our national flag.

Idea 2)

Medical Image Processing with exponential speed up



Picture : Input MRI vs MRI evolving over time

Idea : Given a MRI image, convert it into circuit ,I mean encoding it,then apply unitary operation to simulate evolvment over time with parametarized circuit,then turn evolved circuit into picture.This picture will be matched with a reference image to decide whether the input MRI will transform into a certain state/disease or how likely (% possiblity) the diseases will infect brain.

Effectiveness : In quantum circuit qubit can be in superposition resulting all possible evolution in superposition. We can apply decision oracle simultaneously over all image I mean qubit. Because qubit holds image information. We can bruteforcing all possibilities in reasonably fast time because of superposition and single pass into the decision oracle.

Implementation : Added a function inside given quantumblur library.

```
def mri_quantum_progressive_evolve(image, steps=20, max_angle=np.pi / 2, log=False, save_path="mri_evolve.gif"):
    """
    Evolves a single MRI image using structured, gradual quantum rotation over time.

    Args:
        image (PIL.Image): Input MRI image (RGB).
        steps (int): Number of evolution steps (frames).
        max_angle (float): Max angle of rotation at the final step.
        log (bool): Use logarithmic decoding for circuit-to-height.
        save_path (str): Output path for the generated GIF.

    Returns:
        str: Path to saved animation.
    """
```

```
def evolve_mri_image(mri_image, steps=30, degradation_strength=0.05, noise_strength=0.02, min_intensity=0.2):
    """
    Simulates partial degradation of an MRI image by reducing peak height intensities using quantum circuits,
    ensuring the image does not become fully dark.

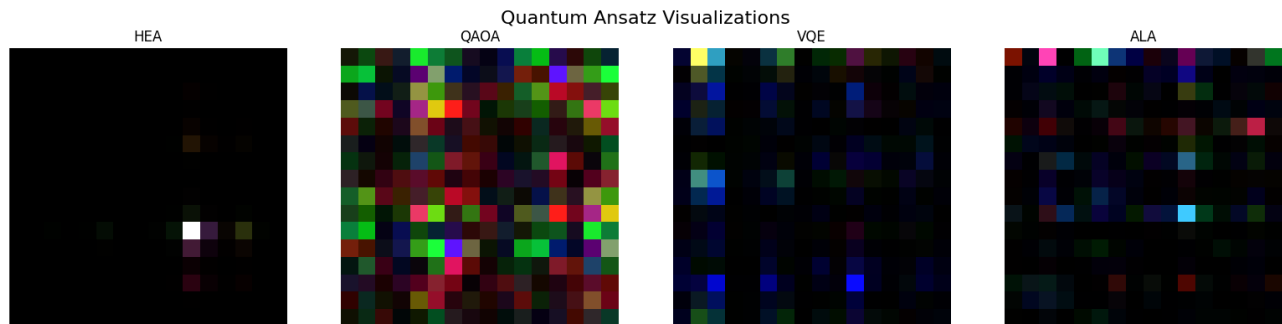
    Args:
        mri_image (PIL.Image): Input MRI image (grayscale, mode 'L').
        steps (int): Number of degradation steps (frames in the animation).
        degradation_strength (float): Maximum angle (in radians) for RX/RV rotations to reduce peak intensities.
        noise_strength (float): Maximum angle (in radians) for random noise rotations to simulate signal loss.
        min_intensity (float): Minimum intensity factor (0 to 1) to prevent full darkness.

    Returns:
        list: List of PIL.Image objects representing the degraded frames, saved as 'mri_degradation.gif'.
    """
```

Tried different time of random evaluation, but medical evolvement is possible with designed parameterized circuits.

Idea 3)

Turn Quantum Circuit into Reality



Idea: We are using different types of ansatz circuits which is turned into image using the provide library.

What is exceptional here? Each ansatz gives unique image under a fixed image generation protocol. So, we can create **QR code** using this that will uniquely identify some inside parameters.

Another use can be visualization the variation in ansatzes. You can consider this as a unique artwork. Also, we can reverse engineering to make ansatz from images which is useful for VQE algo.

Implementation : Added this function inside library

```
def generate_ansatz_image(ansatz_type='HEA', size=(8, 8), layers=3, log=False):  
    """  
    Create a full-color RGB image from a well-known quantum ansatz,  
    with slightly different circuits for each channel.  
    """
```

Idea 4)

Photoshoping



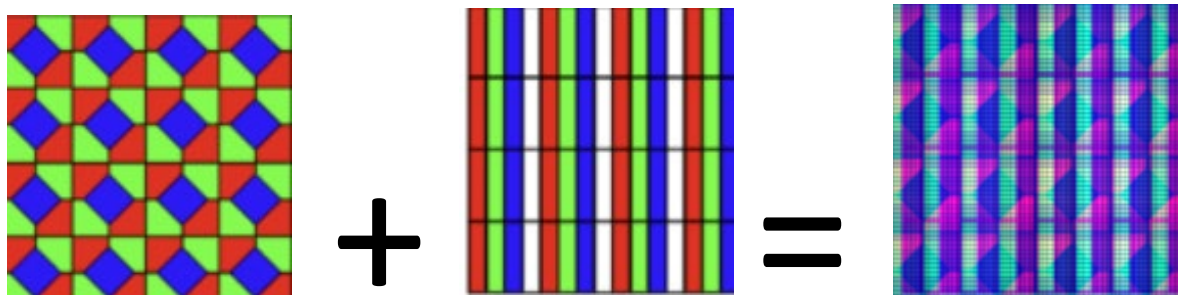
Idea : Applied some rotation gates on Green and Blue part after RGB batch encoding pipeline keeping red intact. Using similar idea we can develop masks and filters.

Implementation: indside library added a new function

```
qb.apply_rgb_rotations(bc,np.pi/2,np.pi)
```

Idea 5)

Photo Merging and Blending



Process : Based on two input image, we create a blend version of these images.

We applied RGB batch encoding, merge R of one image with G of another image and for B starting with fresh Blue we applied some rotation gate, finally decoded image.

Implementation :

```
def merge_rgb_quantum(image1, image2, rotation_angle=np.pi/4, resize=(32, 32), log=False, save_path=None):
    """
    Merges two RGB images by taking R from image1, G from image2, and generating a new B channel
    with random quantum rotations. Applies random RX/RX rotations to R and G channels.

    Args:
        image1 (PIL.Image): Source image for red channel (RGB).
        image2 (PIL.Image): Source image for green channel (RGB).
        rotation_angle (float): Maximum angle (in radians) for RX/RX rotations on R, G, and B.
        resize (tuple): Target size (width, height) for both images to ensure compatibility.
        log (bool): If True, use logarithmic encoding/decoding for quantum circuits.
        save_path (str): Path to save the output image.

    Returns:
        PIL.Image: Merged RGB image with processed R, G, and new B channels.
    """
```




Experiment with various image on the same scheme :

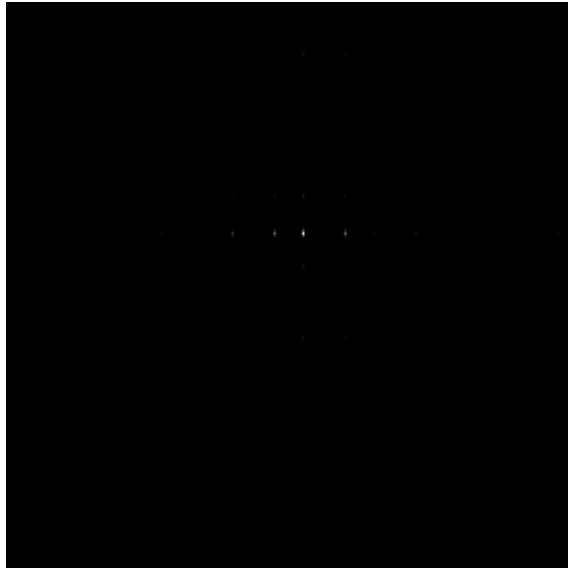
Reference Image: (<https://sites.google.com/view/poulomisarkerpuja/home>) Taken his permission for using this images. So all rights reserved. @



These are results of blending reference images.

Idea 6:

Simulating “Quantum Image” Degradation



Idea : Gradually added noise conditionally applying some phase gates to simulate how image can evolve noisy over time or on explored to real environment .

Implementation:

```
def time_lapse_decay(image, steps=30, max_noise=0.2, max_blur=0.5):
    frames = []
    circuits = image2circuits(image)
    for step in range(steps):
        noise = max_noise * step / steps
        blur = max_blur * step / steps
        for qc in circuits:
            for q in range(qc.num_qubits):
                qc.rx(random.uniform(-noise, noise) * np.pi, q)
                if q < qc.num_qubits - 1 and random.random() < 0.1:
                    qc.cx(q, q + 1)
            circuits = blur_image(image, xi=blur, circuits=circuits)
        frames.append(circuits2image(circuits))
    frames[0].save("decay.gif", save_all=True, append_images=frames[1:], duration=100, loop=0)
    return frames
```


Conclusion :

Played around quantum algorithm and library given, learned a lot of new things, explored a different quantum application field. This was a nice hackathon indeed.

Thanks for arranging such an event.

With Regards,

[Niloy Kumar Mondal](#)

Undergrad CSE Student

[Bangladesh University of Engineering and Technology](#)

Email : 2105044@ugrad.cse.buet.ac.bd

Personal Contact : niloy12101001@gmail.com

Team Leader of **QuittCat**